

# Deadline 5 — Exactly-Once

## What this session set out to do, and what happened

Build the third and last of the system's guarantees: **exactly-once**. A retried GPU reservation must produce one allocation and return the original response, and the idempotency key must commit in the same transaction as the allocation it guards.

That is done and verified. **A'S COLUMN MET** Deadline 5 itself is **STILL OPEN** — its checkpoint names Benchmark 1, which belongs to B's concurrency harness.

**No migration was needed, which was not expected.** Every previous A deadline opened by generating an Alembic revision. `idempotency_keys` and its `UNIQUE(key, user_id)` shipped at Deadline 1 in `705b757e5df2`, and `alembic check` reports *"No new upgrade operations detected."* Deadline 5 turned out to be pure application code. Checking rather than assuming is what stopped an empty revision going into the chain.

## Files created

| File                                                 | Lines | What it is                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app/idempotency/service.py</code> <b>NEW</b>   | 211   | The module. <code>request_fingerprint()</code> , <code>claim()</code> , <code>record_response()</code> , and three exceptions ( <code>KeyReplayed</code> , <code>KeyReuse</code> , <code>KeyInFlight</code> ). Same contract as <code>quotas/</code> : takes a <code>Session</code> it did not create, never commits, knows nothing about HTTP. |
| <code>app/idempotency/__init__.py</code> <b>NEW</b>  | 0     | Package marker, matching <code>quotas/</code> .                                                                                                                                                                                                                                                                                                 |
| <code>scripts/check_idempotency.py</code> <b>NEW</b> | 413   | The sixth verification gate. 37 assertions, exits non-zero on the first failure, leaves the database at its post-seed state.                                                                                                                                                                                                                    |

## Files modified

| File                              | Δ lines  | Change                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app/gpus/service.py MOD           | +82 / -6 | <code>reserve_gpu()</code> takes an optional <code>idempotency_key</code> . The claim goes in at <b>step (1), above both row locks</b> , so a retry that will be replayed never queues behind a real allocation. <code>record_response()</code> runs after <code>flush()</code> and <b>before the existing commit</b> — no second commit was added. Added the <code>RESERVE_ENDPOINT</code> constant. |
| app/gpus/router.py MOD            | +45 / -6 | Optional <code>Idempotency-Key</code> header, plus three new response paths: replay via <code>JSONResponse</code> carrying the stored status, 422 <code>IDEMPOTENCY_KEY_REUSED</code> , and 409 <code>IDEMPOTENCY_IN_PROGRESS</code> .                                                                                                                                                                |
| DECISIONS.md MOD                  | +244     | New section " <i>Deadline 5 — exactly-once (A)</i> ": the index-not-a-lock argument, the savepoint, the three measured builds, and the corrected claim. Plus " <i>Outstanding — B, before Deadline 5 can close</i> ", recording the four items that stand between A's column being met and the deadline being met.                                                                                    |
| WORK_LOG.md MOD                   | +108     | Session 11 entry, with the verification output and seven carried-forward items. States plainly that A's column is met and the deadline is not.                                                                                                                                                                                                                                                        |
| ARCHITECTURE_AND_WORKFLOWS.md MOD | +45 / -4 | \$6 gains the measured broken-vs-fixed table; \$7 fixes the replay status and adds <code>IDEMPOTENCY_IN_PROGRESS</code> ; \$14 corrects the retry rows.                                                                                                                                                                                                                                               |
| EXECUTION_PLAN.md MOD             | +38 / -5 | Deadline 5 marked <i>A's column met</i> , with the two items the original column did not mention recorded against it.                                                                                                                                                                                                                                                                                 |

## The mechanism, in one paragraph

The other two guarantees are row locks. Exactly-once cannot be, and the reason is the whole point: **there is nothing to lock until somebody creates the row, and creating the row is exactly the window a retry arrives in**. The serialization point is the `UNIQUE(key, user_id)` index. Two simultaneous retries race to INSERT; one proceeds and the other *blocks on the index entry* until the first transaction ends, then either sees the violation (it committed — replay its stored response) or succeeds (it rolled back — do the work for real).

|              |                           |    |                   |
|--------------|---------------------------|----|-------------------|
| capacity     | a fact about the RESOURCE | -> | resource row lock |
| quota        | a fact about the USER     | -> | user row lock     |
| exactly-once | a fact about the REQUEST  | -> | unique index      |

## Verification

scripts/check\_idempotency.py — **37/37 PASS**. Full regression after touching the flagship transaction: `check_jwt`, `check_auth`, `check_rbac`, `check_rooms`, `check_gpus`, `check_courses` — all pass. `alembic check` — no drift.

|                                  |    |                                                     |
|----------------------------------|----|-----------------------------------------------------|
| no key, twice                    | -> | 2 reservations, distinct ids                        |
| same key, twice                  | -> | 1 reservation, 201, identical body                  |
| third retry, later               | -> | still replays, still 1 reservation                  |
| same key, different body         | -> | 422 <code>IDEMPOTENCY_KEY_REUSED</code>             |
| same key, different cluster      | -> | 422       <- fingerprint covers <code>gpu_id</code> |
| same key STRING, different users | -> | 2 reservations, 2 key rows                          |
| over quota with a key            | -> | 409, ZERO key rows left behind                      |
| same key after cancelling        | -> | 201, allocates for real                             |
| BENCHMARK 3, 8 retries x 15      | -> | {201: 120}, exactly 1 row per trial                 |

## Broken-vs-fixed: three builds, all actually run

Both broken builds were written and measured before the correct one was kept — this project's standing rule that the broken version is built first, so the table is reproducible rather than reconstructed at the end. 8 simultaneous

retries of one key  $\times$  15 trials = 120 requests.

| Build                                  | 201 | 409 | 500 | rows / trial | seq. failures |
|----------------------------------------|-----|-----|-----|--------------|---------------|
| <b>correct</b> — savepoint, one commit | 120 | 0   | 0   | 1            | 0             |
| check-then-insert, no savepoint        | 34  | 0   | 86  | 1            | 1             |
| key commits separately                 | 22  | 98  | 0   | 1            | 2             |

**The finding that changes what this project claims: no build over-allocated. Not one.** `UNIQUE(key, user_id)` held the row count to exactly one in all three — the database was never going to permit a double-booking, whatever the application did.

So "*idempotency prevents double-booking*" is **false for this system**, and the README was one sentence away from asserting it. What the correct implementation actually buys is that **the retry gets the original response**: without the savepoint the unique violation aborts the transaction and 86 of 120 retries become 500 s; with a split commit the key is visible carrying a NULL response and 98 of 120 become spurious 409 s. The claim has been corrected in both `DECISIONS.md` and `ARCHITECTURE_AND_WORKFLOWS.md`.

## Two things the plan's Deadline 5 column did not mention

- **The SAVEPOINT.** A unique violation aborts the entire Postgres transaction — every statement after it fails with `InFailedSqlTransaction`, including the read that fetches the stored response, which is the entire point of catching the violation. `db.begin_nested()` makes the rollback partial so the surrounding transaction survives. Not a detail: without it the feature does not work under concurrency at all.
- **The Idempotency-Key header must be optional.** Benchmark 3's whole shape is the contrast between sending it and not, so requiring it would delete one column of the table it exists to produce. The unkeyed double-allocation is asserted as a *passing* test for this reason.

## Decisions settled

- **A replay returns 201, not 200.** §7 said 200 and §14 said 200/201 — a contradiction nobody had noticed. It returns the *stored* status, because a replay must be indistinguishable from the original call; a client branching on 201 would otherwise take a different path on the retry, which is the exact bug idempotency removes.
- **Exactly-once is a promise about successes.** A request refused by the quota gate leaves no key — the claim rolls back with everything else — so a later retry allocates for real. The alternative has to answer "for how long is the caller forbidden from retrying a request that never happened?"
- **The fingerprint covers the cluster id, not just the body.** The same body posted to two clusters is two requests. `endpoint` is compared too. `hashlib.sha256` over sorted JSON — Python's `hash()` is salted per process and would break every cross-worker replay while passing every single-process test.

## What is left

- **B is blocked, unchanged.** `tests/` is still empty and `pytest-asyncio` is still absent from `requirements.txt`. Verified this session, not assumed.
- **The connection pool is now the top joint item.** `session.py` sets no `pool_size` — 5 + 10 overflow = 15 against a 500-request harness. Carried since session 5; Benchmark 1 is what it distorts. Shared file, needs both people.
- **Item 11 untouched:** the unexplained stale locked read on the GPU path, still due before Deadline 7's promotion transaction.

- **Items 7, 9, 10** remain joint calls blocking Deadline 7; item 9 (global lock order vs. promotion) is still the urgent one.
- Idempotency is wired into **the GPU path only** — rooms and courses take no key. That matches the plan, but the README should say so rather than leave it to be noticed.
- No expiry on `idempotency_keys`; rows accumulate forever. Fine for a demo — worth a line under "Designed, not implemented."